

Merging `shared_ptr` changes from Library Fundamentals to C++17

Revision History

Changes since P0414R0, following LWG review in Chicago, and LWG telecon:

- For `operator[]` removed `noexcept` and replaced with "*Throws: Nothing.*" This is a change from the TS in order to adhere to the guidelines used for the IS.
- For `operator->` changed the first new paragraph from "When τ is an array type or (possibly cv-qualified) `void`" to the correct wording from P0220R1: "*Remarks: When τ is an array type.*"

Introduction

LWG Motion 6 in Jacksonville was to apply to the C++ working paper the wording from [P0220R1](#), *Adopt Library Fundamentals V2 TS Components for C++17 (R1)*, but due to conflicts with changes in the C++ WP the `shared_ptr` changes were not applied.

I offered to write this paper showing the changes relative to the current WP. I failed to do so before Oulu in time for the CD, but provide it now.

Proposed Wording

Changes are relative to N4594. Clauses have been renumbered since that draft, so cross-references are given using **[stable.names]** for clarity.

Modify the class synopsis in 20.10.2.2 Class template `shared_ptr` [util.smartptr.shared]:

```
namespace std {
    template<class T> class shared_ptr {
    public:

        typedef T element_type;
        using element_type = remove_extent_t<T>;

        [...]

        // 20.10.2.2.5, observers:
        Telement_type* get() const noexcept;
        T& operator*() const noexcept;
        T* operator->() const noexcept;
        element_type& operator[](ptrdiff_t i) const;
        long use_count() const noexcept;
        bool unique() const noexcept;
        explicit operator bool() const noexcept;
        template<class U> bool owner_before(shared_ptr<U> const& b) const;
        template<class U> bool owner_before(weak_ptr<U> const& b) const;
    };

    [...]

    // 20.10.2.2.9, shared_ptr casts:
    template<class T, class U>
        shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
    template<class T, class U>
        shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
    template<class T, class U>
        shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
    template<class T, class U>
        shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
}
```

Add a new paragraph at the end of 20.10.2.2 Class template `shared_ptr` [util.smartptr.shared]:

-5- For the purposes of subclause [util.smartptr], a pointer type Y^* is said to be *compatible with* a pointer type T^* when either Y^* is convertible to T^* or Y is $U[N]$ and T is $U\text{ cv }[]$.

Change 20.10.2.2.1 `shared_ptr` constructors [util.smartptr.shared.const]:

```
template<class Y> explicit shared_ptr(Y* p);
```

-4- ~~Requires: p shall be convertible to T^* .~~ Y shall be a complete type. The expression ~~`delete p`~~ `delete[] p`, when T is an array type, or `delete p`, when T is not an array type, shall be well formed, shall have well defined behavior, and shall not throw exceptions. When T is $U[N]$, $Y(*)[N]$ shall be convertible to T^* ; when T is $U[], Y(*)[]$ shall be convertible to T^* ; otherwise, Y^* shall be convertible to T^* .

-5- ~~Effects: E~~ When T is not an array type, constructs a `shared_ptr` object that owns the pointer p . Otherwise, constructs a `shared_ptr` that owns p and a deleter of an unspecified type that calls `delete[] p`. Enables `shared_from_this` with p . If an exception is thrown, `delete p` is called when T is not an array type, `delete[] p` otherwise.

-6- *Postconditions:* `use_count() == 1` && `get() == p`.

-7- *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory could not be obtained.

```
template<class Y, class D> shared_ptr(Y* p, D d);  
template<class Y, class D, class A> shared_ptr(Y* p, D d, A a);  
template <class D> shared_ptr(nullptr_t p, D d);  
template <class D, class A> shared_ptr(nullptr_t p, D d, A a);
```

-8- ~~Requires: p shall be convertible to T^* .~~ D shall be `CopyConstructible`. The copy constructor and destructor of D shall not throw exceptions. The expression `d(p)` shall be well formed, shall have well defined behavior, and shall not throw exceptions. A shall be an allocator (17.6.3.5). The copy constructor and destructor of A shall not throw exceptions. When T is $U[N]$, $Y(*)[N]$ shall be convertible to T^* ; when T is $U[], Y(*)[]$ shall be convertible to T^* ; otherwise, Y^* shall be convertible to T^* .

-9- *Effects:* Constructs a `shared_ptr` object that owns the object p and the deleter d . The first and second constructors enable `shared_from_this` with p . The second and fourth constructors shall use a copy of a to allocate memory for internal use. If an exception is thrown, `d(p)` is called.

-10- *Postconditions:* `use_count() == 1` && `get() == p`.

-11- *Throws:* `bad_alloc`, or an implementation-defined exception when a resource other than memory could not be obtained.

```
template<class Y> shared_ptr(const shared_ptr<Y>& r,  $T$ element_type* p) noexcept;
```

-12- *Effects:* [...]

-13- *Postconditions:* [...]

-14- *[Note:* [...]

-15- *[Note:* [...]

```
shared_ptr(const shared_ptr& r) noexcept;  
template<class Y> shared_ptr(const shared_ptr<Y>& r) noexcept;
```

-16- *Remark:* The second constructor shall not participate in overload resolution unless Y^* is ~~implicitly convertible to~~ *compatible with* T^* .

-17- *Effects:* [...]

-18- *Postconditions:* [...]

```
shared_ptr(shared_ptr&& r) noexcept;  
template<class Y> shared_ptr(shared_ptr<Y>&& r) noexcept;
```

-19- *Remark:* The second constructor shall not participate in overload resolution unless Y^* is ~~convertible to~~ *compatible with* T^* .

-20- *Effects:* [...]

-21- *Postconditions:* [...]

```
template<class Y> explicit shared_ptr(const weak_ptr<Y>& r);
```

-22- Requires: T^* shall be ~~convertible to~~ compatible with T^* .

-23- Effects: [...]

-24- Postconditions: [...]

-25- Throws: [...]

```
template <class Y, class D> shared_ptr(unique_ptr<Y, D>&& r);
```

-26- Remark: This constructor shall not participate in overload resolution unless `unique_ptr<Y, D>::pointer` is ~~convertible to~~ compatible with T^* .

-27 Effects: [...]

Change 20.10.2.2.5 `shared_ptr` observers [util.smartptr.shared.obs]:

```
Telement_type* get() const noexcept;
```

-1- Returns: the stored pointer.

```
T& operator*() const noexcept;
```

-2- Requires: `get() != 0`.

-3- Returns: `*get()`.

-4- Remarks: When T is an array type or (possibly cv-qualified) `void`, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

```
T* operator->() const noexcept;
```

-5- Requires: `get() != 0`.

-6- Returns: `get()`.

-?- Remarks: When T is an array type, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

```
element_type& operator[](ptrdiff_t i) const;
```

-?- Requires: `get() != 0` && `i >= 0`. If T is $U[N]$, `i < N`.

-?- Returns: `get()[i]`.

-?- Remarks: When T is not an array type, it is unspecified whether this member function is declared. If it is declared, it is unspecified what its return type is, except that the declaration (although not necessarily the definition) of the function shall be well formed.

-?- Throws: Nothing.

Change 20.10.2.2.9 `shared_ptr` casts [util.smartptr.shared.cast]:

```
template<class T, class U> shared_ptr<T> static_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-1- Requires: The expression `static_cast<T*>(r.get()(U*)0)` shall be well formed.

-2- Returns: `shared_ptr<T>(r, static_cast<typename shared_ptr<T>::element_type*>(r.get()))`. ~~If r is empty, an empty `shared_ptr<T>`; otherwise, a `shared_ptr<T>` object that stores `static_cast<T*>(r.get())` and shares ownership with r .~~

~~-3- Postconditions: `w.get() == static_cast<T*>(r.get())` and `w.use_count() == r.use_count()`, where w is the return value.~~

-4- [Note: The seemingly equivalent expression `shared_ptr<T>(static_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. --end note]

```
template<class T, class U> shared_ptr<T> dynamic_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-5- Requires: The expression `dynamic_cast<T*>(r.get()(U*)0)` shall be well formed and shall have well defined behavior.

-6- *Returns:*

- When `dynamic_cast<T>::element_type*>(r.get())` returns a nonzero value, ~~a `shared_ptr<T>` object that stores a copy of it and shares ownership with `r`~~, `shared_ptr<T>(r, p)`;
- Otherwise, ~~an empty `shared_ptr<T>` object.~~

~~-7- *Postcondition:* `w.get() == dynamic_cast<T*>(r.get())`, where `w` is the return value.~~

-8- [Note: The seemingly equivalent expression `shared_ptr(dynamic_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. *--end note*]

```
template<class T, class U> shared_ptr<T> const_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-9- *Requires:* The expression `const_cast<T*>(r.get())(U*)0` shall be well formed.

-10- *Returns:* `shared_ptr<T>(r, const_cast<typename shared_ptr<T>::element_type*>(r.get()))`. ~~If `r` is empty, an empty `shared_ptr<T>`; otherwise, a `shared_ptr<T>` object that stores `const_cast<T*>(r.get())` and shares ownership with `r`.~~

~~-11- *Postconditions:* `w.get() == const_cast<T*>(r.get())` and `w.use_count() == r.use_count()`, where `w` is the return value.~~

-12- [Note: The seemingly equivalent expression `shared_ptr<T>(const_cast<T*>(r.get()))` will eventually result in undefined behavior, attempting to delete the same object twice. *--end note*]

```
template<class T, class U> shared_ptr<T> reinterpret_pointer_cast(const shared_ptr<U>& r) noexcept;
```

-?- *Requires:* The expression `reinterpret_cast<T*>((U*)0)` shall be well formed.

-?- *Returns:* `shared_ptr<T>(r, reinterpret_cast<typename shared_ptr<T>::element_type*>(r.get()))`.

Change 20.10.2.3.1 `weak_ptr` constructors [util.smartptr.weak.const]:

```
weak_ptr(const weak_ptr& r) noexcept;  
template<class Y> weak_ptr(const weak_ptr<Y>& r) noexcept;  
template<class Y> weak_ptr(const shared_ptr<Y>& r) noexcept;
```

-3- *Remark:* The second and third constructors shall not participate in overload resolution unless `Y*` is ~~implicitly convertible to~~ compatible with `T*`.

Acknowledgments

Thanks to Alisdair Meredith for his careful review which caught some accidental discrepancies from P0220R1.